Mixed-Integer Programming Formulation and Implementation of the T-ASR Solver

Contents

1 Overview **3**

2 ECMP split-coefficient preprocessing **3**

3 Sets and notation **3**

4 Parameters and numerical tolerances **4**

5 Decision variables **4**

6 Base formulation **5**
- 6.1 Exact arc-utilization equations . . . 5
- 6.2 Flow conservation . . . 5
- 6.3 Node-degree inequalities . . . 5
- 6.4 Maximum number of segments . . . 5
- 6.5 Reconfiguration budget . . . 6
- 6.6 Variable domains . . . 6

7 Sparse preprocessing and variable bounds **6**
- 7.1 Reachability preprocessing . . . 6
- 7.2 Global maximum-load pruning . . . 6
- 7.3 Load bounds during the solve . . . 7

8 Lexicographic objective **7**
- 8.1 First iteration: global min–max . . . 7
- 8.2 Persistent bottleneck peeling . . . 7
- 8.3 Stopping peeling and switching to the residual formulation . . . 8
- 8.4 Residual ordered-k-sum formulation . . . 8

9 Initial common-routing warm start **9**

10 Warm starts and persistent model reuse **9**

11 Model size **9**

12 Global time limit, interruption, and incumbent selection **10**

13 Solver-independent backend **10**

14 LP model export **10**

15 Solution extraction and output **10**

16 Program input and command-line interface **10**

17 Windows development setup **11**

18 Compact interpretation **11**

1 Overview

This project implements an experimental mixed-integer programming approach to the T-Adaptive Segment Routing (T-ASR) problem of the ROADEF/EURO 2026 "Keep the Flow!" challenge. The implementation first computes ECMP split coefficients for every time period, segment, and available network arc. It then constructs a sparse mixed-integer linear model whose routing decisions select segment paths for every demand and time period.

The intended objective is lexicographic minimization of the vector of arc utilizations over all available arc–time pairs. The first level minimizes the maximum utilization. Subsequent levels are handled by a persistent bottleneck peeling procedure whenever individual bottlenecks can be proved unavoidable. If that proof fails, the implementation switches to a residual ordered-k-sum formulation over the still-unfixed loads.

The mathematical model is represented through a solver-independent sparse linear-model interface. The same formulation can be solved with HiGHS or the Gurobi C API. The implementation also includes sparse preprocessing, an optional common-routing warm start, persistent native solver sessions, MIP starts for rebuilt residual models, global time-limit handling, graceful interruption, physical-load consistency checks, LP export, and JSON solution extraction.

2 ECMP split-coefficient preprocessing

The split coefficients are computed before the MILP is built. For each time period, the implementation copies the network arc metrics and sets the metric of every intervened arc to infinity. For each destination node, reverse Dijkstra then computes shortest-path distances in the time-dependent network.

For an arc $a = (u, v)$ and destination r, the arc is eligible for forwarding toward r when

$$d_t(u, r) \approx w_a^{(t)} + d_t(v, r),$$

where $d_t(\cdot, r)$ is the shortest-path distance and $w_a^{(t)}$ is the arc metric at time t. For every reachable source–destination pair, one unit of flow is initialized at the source. At each node, the incoming flow is split equally among all eligible forwarding arcs, and the resulting arc flows are stored as the coefficients $r_{i,j,a,t}$. Only positive coefficients are kept in the sparse representation.

This preprocessing models equal-cost multipath (ECMP) forwarding and makes the coefficients fixed parameters of the subsequent optimization model.

3 Sets and notation

Let

$$G = (V, A)$$

be the directed network. Let

$$T = \{0, 1, \ldots, h-1\}, \qquad T^* = T \setminus \{0\},$$

where T is the set of time periods. For every $t \in T$, let $q(t) \subseteq A$ be the set of unavailable arcs and define

$$A_t := A \setminus q(t).$$

Each demand $d \in D$ has a source node s_d and a target node t_d. The demand volume at time t is denoted by $\nu_{d,t}$.

For a segment (i, j) and time t, the implementation computes sparse ECMP split coefficients

$$r_{i,j,a,t} \geq 0,$$

where $r_{i,j,a,t}$ is the fraction of the flow associated with segment (i,j) that traverses arc a at time t. Intervened arcs are assigned infinite forwarding metric and consequently do not receive positive split coefficients.

Define the set of reachable segments at time t as

$$P_t := \{(i,j) \in V \times V : \ i \neq j, \ r_{i,j,a,t} > 0 \text{ for at least one } a \in A_t\}.$$

Only segments in P_t are considered when creating routing variables.

The set of available arc–time pairs is

$$\mathcal{A} := \{(t,a) : t \in T, \ a \in A_t\}, \qquad m := |\mathcal{A}|.$$

The m corresponding utilization values are the components of the lexicographic objective.

4 Parameters and numerical tolerances

Parameter	**Description**
c_a	Capacity of arc $a \in A$.
$r_{i,j,a,t}$	ECMP split coefficient for segment (i,j), arc a, and time t.
$\nu_{d,t}$	Traffic volume of demand d at time t.
κ_t	Maximum total number of segment-selection changes between $t-1$ and t.
maxSeg	Maximum number of segments in one demand path.
U	Optional maximum-load upper bound obtained from the common-routing warm-start model.
L_k	k-th ordered utilization level, with $L_1 \geq L_2 \geq \cdots$.
S_k	Cumulative ordered objective, $S_k = \sum_{j=1}^{k} L_j$.
R_q	Cumulative ordered objective of order q on the residual unfixed-load vector.
ε_{lex}	Numerical tolerance used for preserving/fixing lexicographic levels; the implementation uses 10^{-6}.
τ	Bottleneck proof/early-rejection tolerance; the implementation uses 10^{-7}.

The code also uses smaller tolerances for diagnostics and segment fixing. The load equations are checked against physical utilization recomputed directly from the selected binary routing variables.

5 Decision variables

Variable	**Description**
$x_{i,j}^{d,t} \in \{0,1\}$	One if segment (i,j) is selected in the path of demand d at time t. It is created only for reachable segments that survive global maximum-load preprocessing.
$\lambda_{a,t} \in \mathbb{R}_{\geq 0}$	Exact utilization of available arc a at time t. Its load equation is an equality, not an inequality.

Variable	Description
$y_{i,j}^{d,t} \in [0,1]$	Auxiliary variable for the absolute difference between the segment-selection variables at times $t-1$ and t. It is created only when the segment exists in at least one adjacent period.
$z \in \mathbb{R}_{\geq 0}$	Maximum-load variable used by the first and all persistent peeled min–max iterations. The same column is reused; no new z_{free} variable is created for each peeled level.
$\alpha_i \in \mathbb{R}$	Threshold variable for residual ordered-sum level $i \geq 2$.
$\beta_p^i \in \mathbb{R}_{\geq 0}$	Excess variable for residual free-load position p and residual level $i \geq 2$. Fixed bottleneck loads are not included in these auxiliary variables.

6 Base formulation

The following constraints are present in every full T-ASR model. Terms for variables that were not created during preprocessing are omitted.

6.1 Exact arc-utilization equations

For every $(t,a) \in \mathcal{A}$, the utilization is defined by

$$c_a \lambda_{a,t} - \sum_{d \in D} \sum_{(i,j) \in P_t} r_{i,j,a,t}\, \nu_{d,t}\, x_{i,j}^{d,t} = 0. \tag{1}$$

Thus $\lambda_{a,t}$ is the physical utilization induced by the routing variables. The equality is important for the later bottleneck tests: a load variable cannot retain artificial slack above the utilization implied by x.

6.2 Flow conservation

For every node, demand, and time period,

$$\sum_{j:(i,j) \in P_t} x_{i,j}^{d,t} - \sum_{j:(j,i) \in P_t} x_{j,i}^{d,t} = \begin{cases} 1, & i = s_d, \\ -1, & i = t_d, \\ 0, & \text{otherwise}, \end{cases} \qquad \forall i \in V,\ d \in D,\ t \in T. \tag{2}$$

6.3 Node-degree inequalities

The formulation limits branching and strengthens the path model through

$$\sum_{j:(i,j) \in P_t} x_{i,j}^{d,t} \leq 1, \qquad \forall i \in V,\ d \in D,\ t \in T, \tag{3a}$$

$$\sum_{j:(j,i) \in P_t} x_{j,i}^{d,t} \leq 1, \qquad \forall i \in V,\ d \in D,\ t \in T. \tag{3b}$$

The final solution extractor additionally rejects branching, cycles, and selected segments that do not form a valid source-to-target path.

6.4 Maximum number of segments

For every demand and time period,

$$\sum_{(i,j) \in P_t} x_{i,j}^{d,t} \leq \text{maxSeg}, \qquad \forall d \in D,\ t \in T. \tag{4}$$

6.5 Reconfiguration budget

For $t \in T^*$, the intended budget is

$$\sum_{d \in D} \sum_{\substack{i,j \in V \\ i \neq j}} \left| x_{i,j}^{d,t} - x_{i,j}^{d,t-1} \right| \leq \kappa_t. \tag{5}$$

A segment variable missing in one of the two periods is interpreted as zero. The implementation linearizes (5) with

$$y_{i,j}^{d,t} \geq x_{i,j}^{d,t} - x_{i,j}^{d,t-1}, \tag{5a}$$

$$y_{i,j}^{d,t} \geq x_{i,j}^{d,t-1} - x_{i,j}^{d,t}, \tag{5b}$$

$$\sum_{d \in D} \sum_{\substack{i,j \in V \\ i \neq j}} y_{i,j}^{d,t} \leq \kappa_t. \tag{5c}$$

Since the x variables are binary and $0 \leq y \leq 1$, these constraints make the budget count the required changes.

6.6 Variable domains

$$x_{i,j}^{d,t} \in \{0, 1\}, \tag{6a}$$

$$\lambda_{a,t} \in \mathbb{R}_{\geq 0}, \tag{6b}$$

$$0 \leq y_{i,j}^{d,t} \leq 1. \tag{6c}$$

7 Sparse preprocessing and variable bounds

7.1 Reachability preprocessing

The implementation does not create all potential $|D||T||V|(|V|-1)$ segment variables. A segment (i, j) is omitted at time t if its sparse split-coefficient list is empty. This removes segments that cannot forward traffic in that time period.

7.2 Global maximum-load pruning

If a finite incumbent upper bound U is available, selecting segment (i, j) for demand d at time t alone induces at least

$$\ell_{d,t,i,j} := \max_{a : r_{i,j,a,t} > 0} \frac{r_{i,j,a,t} \nu_{d,t}}{c_a}. \tag{7}$$

If

$$\ell_{d,t,i,j} > U + \varepsilon_{\text{lex}},$$

the corresponding x variable is not created because it cannot occur in a solution whose global maximum utilization is at most U.

The pruning cutoff must always be a valid bound on the *global* maximum L_1, not the smaller maximum of the currently unfixed residual loads. A segment may contribute more than a later residual boundary to an already fixed earlier bottleneck. After the first full iteration, the optimal value L_1 is used to fix additional existing segment variables to zero without rebuilding the persistent model.

7.3 Load bounds during the solve

The initial full model can use $U + \varepsilon_{\text{lex}}$ as an upper bound on its load variables and also adds $z \le U$ when U is available. During a persistent bottleneck test, every still-free load is temporarily bounded by the current simple-max boundary plus ε_{lex}. Proven bottleneck loads retain a narrow interval around their fixed value.

When the residual formulation is activated, the still-free loads receive the explicit upper bound $R_1 + \varepsilon_{\text{lex}}$, while fixed bottlenecks retain their own narrow bounds. The global L_1 cutoff continues to be used for segment pruning.

8 Lexicographic objective

Let the m utilization values be sorted in non-increasing order:

$$L_1 \ge L_2 \ge \cdots \ge L_m, \qquad S_k := \sum_{j=1}^{k} L_j.$$

The target is lexicographic minimization of $(L_1, L_2, \ldots, L_m)$. The implementation solves this sequentially, retaining the cumulative values S_k. The ordered component can be recovered from

$$L_1 = S_1, \qquad L_k = S_k - S_{k-1} \quad (k \ge 2).$$

8.1 First iteration: global min–max

The first iteration minimizes the maximum load:

$$\begin{aligned}
S_1 := \min \quad & z && && (1)\\
\text{s.t.} \quad & z \ge \lambda_{a,t}, && \forall (t,a) \in \mathcal{A}, && (8a)\\
& z \le U, && \text{if a finite warm-start bound is available}, && (8b)\\
& \text{constraints (1)–(6).}
\end{aligned}$$

The cutoff in (8b) is optional and is only an incumbent bound; correctness does not depend on the common-routing warm start succeeding.

8.2 Persistent bottleneck peeling

After each simple-max iteration, the implementation selects a candidate from the *physical* utilizations recomputed directly from the x values. The candidate is the largest still-unfixed load, rather than necessarily the largest reported load variable in a model that might contain slack.

Suppose the current simple-max boundary is L_k and the previously proved bottlenecks are stored in a set F. The bottleneck test is performed on the same native solver session and does not reconstruct the base model:

1. Every still-free load is temporarily bounded by $L_k + \varepsilon_{\text{lex}}$. Already fixed loads keep their narrow fixed-value bounds.
2. The objective coefficient of the persistent maximum variable z is changed from 1 to 0, and the coefficient of the candidate load $\lambda_{\hat{a},\hat{t}}$ is changed to 1. A temporary objective link variable is not needed.
3. The candidate load is minimized. A backend callback may stop as soon as it finds a feasible candidate strictly below $L_k - \tau$. Such an incumbent already disproves unavoidability, so a full optimality proof is unnecessary in that case.
4. If the helper solve is optimal and its minimum is at least $L_k - \tau$, the candidate is declared individually unavoidable. If it can be reduced, or if the helper solve is interrupted or otherwise not proven optimal, no fixing is performed.

The candidate is checked in three ways: the temporary objective value must agree with the candidate λ value, the candidate λ must agree with the physical utilization recomputed from x, and the helper optimum must not exceed the feasible current boundary by more than the diagnostic tolerance.

If a candidate is proved unavoidable, the implementation:

1. fixes its load variable to a narrow interval around L_k;
2. restores the objective to minimization of z; and
3. removes the candidate's λ coefficient from its persistent max-load row. The row $z - \lambda_{\hat{a},\hat{t}} \geq 0$ thereby becomes $z \geq 0$, so the fixed load is peeled out of the next max objective while the row and column numbering remain unchanged.

The same native model is then re-optimized for the next level. Consequently, peeled iterations reuse the original rows, columns, presolve information, and solver state as far as the backend permits. No new max variable is added for each peeled level.

8.3 Stopping peeling and switching to the residual formulation

Let m_f be the number of bottleneck loads that were proved individually unavoidable when peeling stops. The simple-max iteration that detects the failed proof has already optimized the first residual level

$$R_1 = L_{m_f+1}.$$

The corresponding prefix objective is

$$S_{m_f} = \sum_{j=1}^{m_f} L_j,$$

with $S_0 = 0$. The next iteration is therefore not a global ordered-k-sum model; it is a residual ordered-sum model over only the still-unfixed loads.

8.4 Residual ordered-k-sum formulation

Let $\mathcal{F}$ denote the positions of the still-unfixed load variables, and let $q = k - m_f$ be the residual order for global iteration k. The residual level R_1 is preserved directly by the explicit bounds

$$\lambda_p \leq R_1 + \varepsilon_{\text{lex}}, \qquad p \in \mathcal{F}.$$

For $q \geq 2$, the model introduces α_i and β_p^i only for $i = 2, \ldots, q$ and $p \in \mathcal{F}$. The current residual objective is

$$\begin{aligned}
R_q := \min \quad & q\alpha_q + \sum_{p \in \mathcal{F}} \beta_p^q && && (9)\\
\text{s.t.} \quad & \beta_p^i \geq \lambda_p - \alpha_i, && p \in \mathcal{F},\ i = 2, \ldots, q, && (10)\\
& \beta_p^i \geq 0, && p \in \mathcal{F},\ i = 2, \ldots, q, && (11)\\
& i\alpha_i + \sum_{p \in \mathcal{F}} \beta_p^i \leq R_i + \varepsilon_{\text{lex}}, && i = 2, \ldots, q-1, && (12)\\
& \alpha_i \in \mathbb{R}, && i = 2, \ldots, q. && (13)
\end{aligned}$$

Fixed bottleneck loads are excluded from the α/β representation. Only the current residual level q receives objective coefficients; earlier residual levels are preserved by (12). The global cumulative objective and current ordered component are reconstructed as

$$S_k = S_{m_f} + R_q, \qquad L_k = S_k - S_{k-1}.$$

This residual formulation avoids repeatedly introducing auxiliary variables for already fixed bottleneck loads and is smaller than a global ordered-k-sum model.

9 Initial common-routing warm start

Before the main lexicographic sequence, the implementation optionally solves a smaller auxiliary model with one routing decision

$$\bar{x}^d_{i,j}$$

per demand and segment, shared by all time periods. A segment is eligible only if it is reachable in every time period. Loads remain time-dependent and are computed separately for every $(t, a) \in \mathcal{A}$ using the same exact load equations as the full model.

The auxiliary model uses flow conservation, incoming/outgoing degree limits, and the maximum-segment constraint once per demand. It minimizes its maximum load. If a feasible solution is found, its routing is copied to every time period as a MIP start for the first full model, and its measured maximum load provides the valid bound U. If the auxiliary model fails, the full model is solved without this warm start and without the associated cutoff.

Because the route is common to all time periods, it causes zero routing changes in the reconfiguration constraints of the full model.

10 Warm starts and persistent model reuse

The first full model can receive the common-routing solution as a partial MIP start. While bottleneck peeling is active, subsequent iterations are solved by re-optimizing the same native solver session. The code changes column bounds, objective coefficients, and one matrix coefficient per peeled bottleneck instead of rebuilding the base model.

Once peeling has stopped, the residual models are rebuilt. Routing, load, and reconfiguration values from the previous feasible solution are copied by semantic position, so changes in column numbering caused by preprocessing do not invalidate the warm start. For new residual auxiliary variables, the previous free-load vector is sorted and a threshold is chosen from the corresponding order statistic. The initialization is

$$\beta^i_p = \max\{0, \lambda_p - \alpha_i\}.$$

The solver remains free to improve this initialization.

11 Model size

Let X and Y be the sets of actually created segment and reconfiguration variables, respectively, and let $m = |\mathcal{A}|$.

The initial full model contains $|X|$ binary variables, m utilization variables, $|Y|$ reconfiguration variables, and one maximum-load variable z. During persistent peeling, no new columns are added. A proved bottleneck is represented by a narrow load-variable bound and by removing one coefficient from its existing max row.

If peeling stops after m_f bottlenecks have been fixed, let n_f be the number of remaining free load positions and let q be the residual order. A residual model of order $q \geq 2$ introduces

$$(q-1)(n_f+1)$$

additional continuous variables: one α_i and n_f excess variables for each residual level $i = 2, \ldots, q$. No additional binary variables are introduced by the lexicographic extensions.

Without sparse preprocessing, the segment-variable count is bounded by $|D||T||V|(|V|-1)$ and the reconfiguration-variable count by $|D||T^*||V|(|V|-1)$.

12 Global time limit, interruption, and incumbent selection

The command-line time limit is one global wall-clock budget for the complete solve process. It includes the common-routing warm-start attempt, model construction, all persistent re-optimizations, all bottleneck tests, and all rebuilt residual models. Before each backend call, the remaining time is passed to the solver.

If a time limit or a single Ctrl+C interruption prevents an iteration from finishing, a feasible incumbent returned by that iteration is retained whenever one exists. If no incumbent is returned, the best previously retained feasible solution is used. The process compares feasible candidates using their actual load vectors: both vectors are sorted in non-increasing order and compared lexicographically up to the numerical tolerance. This allows a feasible incumbent from an interrupted iteration to replace the last completed lexicographic solution when it is better according to the intended load-vector ordering.

A feasible common-routing solution can serve as a fallback if the main model has not yet produced a full-model incumbent.

13 Solver-independent backend

The formulation is assembled in a generic sparse linear-model representation containing variable names, objective coefficients, bounds, integrality types, and sparse ranged rows. The representation is translated at runtime to either backend:

- **HiGHS**: translated through the HiGHS C++ API;
- **Gurobi**: translated through the Gurobi C API.

The persistent backend interface supports changing column bounds, objective coefficients, row bounds, and individual matrix coefficients. Both backends also support the interrupt callback. The bottleneck helper uses an optional callback criterion that stops minimization as soon as a feasible incumbent below the rejection threshold is found.

The solver is selected at runtime with `--solver highs` or `--solver gurobi`; `grb` is accepted as an alias for Gurobi.

14 LP model export

With `--write-models 1`, the implementation exports the models passed to the selected backend. Filenames include the solver and iteration information. The common-routing model, persistent lexicographic models, bottleneck-test re-optimizations, and residual ordered-sum models can be exported when the corresponding stage is executed. The exported files are useful for inspecting the exact bounds, coefficients, and variable names seen by the backend.

15 Solution extraction and output

For each demand and time period, selected binary segment variables define a successor relation. Starting at the source, the extractor follows successors until the target is reached and records the intermediate nodes as waypoints. It rejects branching, cycles, missing successors, and paths exceeding the maximum number of segments.

The resulting paths are written to the requested `*-srpaths.json` file. Writing is also performed after a global timeout or graceful Ctrl+C whenever a feasible incumbent is available.

16 Program input and command-line interface

For an instance named `setA-01`, the program expects

```
instances/setA-01-net.json
instances/setA-01-tm.json
instances/setA-01-scenario.json
```

The split coefficients are computed from these files before model construction.

A typical invocation is

```
./build/solver.exe \\
  --instance setA-01 \\
  --output setA-01-srpaths.json \\
  --iterations 10 \\
  --time 590 \\
  --solver highs \\
  --write-models 0 \\
  --model-prefix debug-setA-01
```

The relevant options are:

Option	Meaning
`--instance NAME`	Instance basename.
`--output FILE`	Output JSON file containing the segment paths.
`--iterations N`	Maximum number of lexicographic iterations.
`--time SECONDS`	Global time budget for the complete solution process.
`--solver highs|gurobi`	Select the optimization backend at runtime.
`--write-models 0|1`	Disable or enable LP export.
`--model-prefix PREFIX`	Prefix used for exported model files.

17 Windows development setup

The current development setup uses **MSYS2 UCRT64**, CMake, C++20, `networktools`, HiGHS installed through vcpkg for the MinGW triplet, and optionally Gurobi. The project must be configured from the UCRT64 shell so that an UCRT64 compiler is used.

For the HiGHS configuration used during development, the following is an example command sequence:

```
rm -rf build
cmake -B build \\
  -G "MinGW Makefiles" \\
  -DCMAKE_BUILD_TYPE=Release \\
  -DCMAKE_TOOLCHAIN_FILE=C:/vcpkg/scripts/buildsystems/vcpkg.cmake \\
  -DVCPKG_TARGET_TRIPLET=x64-mingw-dynamic
cmake --build build -j
```

The optional Gurobi backend uses the Gurobi C API. Set `GUROBI_HOME` to the installed `win64` directory and ensure that its `bin` directory is on the MSYS2 `PATH`, for example:

```
export GUROBI_HOME="/c/gurobi1302/win64"
export PATH="$GUROBI_HOME/bin:$PATH"
```

18 Compact interpretation

The implementation minimizes the sorted arc–time utilization vector in a sequence of MILPs. The first model minimizes the global maximum load. While possible, bottleneck peeling proves

individual loads unavoidable by minimizing one physical candidate under the current boundary, then fixes proven loads and reuses the same native model for the next min–max level. If a proof fails, the already optimized first residual maximum is preserved directly and later levels are solved with a residual ordered-k-sum formulation that excludes fixed loads. All load equations are exact, and physical-load diagnostics are used to check the consistency of the solver model and the selected routing.